BANKING MANAGEMENT SYSTEM

A Java-Based Web Application

Project Report

Submitted in partial fulfillment of
Java Programming Lab (25BTPCP405)

Submitted By:

Omkar Manoj Bhagat
Roll No: DW236
Class: DW1 | Batch: DW1-2

Academic Year 2024-2025

TABLE OF CONTENTS

ABSTRACT 5
1. INTRODUCTION 6
1.1 Project Overview 6
1.2 Problem Statement 6
1.3 Objectives 6
1.4 Scope of the Project 6
1.4.1 Functional Scope 6
1.4.2 Technical Scope 7
2. SYSTEM ANALYSIS 7
2.1 Requirements Analysis 7
2.1.1 Functional Requirements 7
2.1.2 Non-Functional Requirements 7
2.2 Feasibility Study 7
2.2.1 Technical Feasibility 7
2.2.2 Economic Feasibility 8
2.2.3 Operational Feasibility 8
3. SYSTEM DESIGN 8
3.1 System Architecture 8
3.1.1 Presentation Layer 8
3.1.2 Business Logic Layer 8
3.1.3 Data Access Layer 9
3.2 Database Design 9
3.2.1 Users Table 9
3.2.2 Accounts Table 9
3.2.3 Transactions Table 10
3.2.4 Fraud Alerts Table 10
4. JAVA CONCEPTS IMPLEMENTATION 10
4.1 Unit I: Java Programming Basics 10
4.2 Unit II: Object-Oriented Programming 11
4.2.1 Encapsulation 11
4.2.2 Inheritance 11
4.2.3 Polymorphism 11

4.2.4 Abstraction 11
4.3 Unit III: Interfaces, Packages, and Exception Handling 11
4.3.1 Interfaces 11
4.3.2 Packages 11
4.3.3 Exception Handling 12
4.4 Unit IV: Collections, Multithreading, and Networking 12
4.4.1 Collections Framework 12
4.4.2 Multithreading 12
4.4.3 Networking 12
4.5 Unit V: JDBC Database Connectivity 13
5. FEATURES AND FUNCTIONALITY 13
5.1 User Authentication System 13
5.2 Account Management 13
5.3 Transaction Processing 13
5.3.1 Deposit 13
5.3.2 Withdrawal 14
5.3.3 Fund Transfer 14
5.4 Fraud Detection System 14
5.5 Transaction History and Reporting 14
6. IMPLEMENTATION DETAILS 15
6.1 Development Environment 15
6.2 Project Structure 15
6.3 API Endpoints 15
6.4 Security Measures 15
7. TESTING AND VALIDATION 16
7.1 Testing Methodology 16
7.1.1 Unit Testing 16
7.1.2 Integration Testing 16
7.1.3 System Testing 16
7.2 Test Results 16
8. FUTURE ENHANCEMENTS 17
8.1 Enhanced Account Creation Process 17
8.2 Advanced Input Validation 17
8.2.1 Name Validation 17

8.2.2 Email Verification with OTP 17
8.2.3 Mobile Number Verification 17
8.3 Proactive Fraud Prevention System 17
8.3.1 Transaction Confirmation Workflow 18
8.3.2 Timeout Mechanism 18
8.3.3 Implementation Approach 18
8.4 Local Database Synchronization 18
8.5 Additional Planned Features 19
9. CONCLUSION 20
10. REFERENCES 21

ABSTRACT

The Banking Management System is a comprehensive web-based application developed using Java programming language that simulates real-world banking operations. This project demonstrates the practical implementation of core Java concepts including Object-Oriented Programming, Exception Handling, Multithreading, JDBC database connectivity, and Network programming.

The system provides a complete banking solution with features including user authentication, multiple account types (Savings and Current accounts), transaction management (deposit, withdrawal, and transfer operations), real-time fraud detection using multithreading, and comprehensive transaction history tracking. The application uses Supabase as its PostgreSQL database backend and implements a RESTful API architecture using Java's built-in HTTP server.

A key highlight of this project is the intelligent fraud detection system that employs multithreading to analyze transactions in real-time. The system monitors for suspicious patterns such as large transactions, rapid successive transactions, unusual timing, and repeated round amounts. This feature demonstrates the practical application of concurrent programming and enhances the security of the banking system.

The frontend interface is built using HTML5, CSS3, and JavaScript, providing a responsive and user-friendly experience. The project successfully implements all major Java programming concepts covered in the syllabus including inheritance, polymorphism, interfaces, packages, exception handling, collections framework, and networking, making it a comprehensive demonstration of Java application development skills.

1. INTRODUCTION

1.1 Project Overview

The Banking Management System is a full-stack web application that replicates core banking functionalities in a digital environment. In today's digital age, online banking has become an essential service, and this project aims to understand and implement the fundamental operations that power modern banking systems.

This project serves as a practical implementation of the Java programming concepts learned throughout the course. It demonstrates how theoretical concepts such as inheritance, polymorphism, exception handling, and multithreading can be applied to build a real-world application that solves actual business problems.

1.2 Problem Statement

Traditional banking systems require customers to visit physical branches for various transactions, which is time-consuming and inefficient. Moreover, manual transaction processing increases the risk of errors and makes it difficult to detect fraudulent activities in real-time. There is a need for an automated system that can:

- Provide secure user authentication and authorization
- Allow users to manage multiple account types with different features
- Process transactions quickly and accurately
- Detect and prevent fraudulent activities in real-time
- Maintain comprehensive transaction history and audit trails

1.3 Objectives

The primary objectives of this project are:

1. To develop a secure and efficient banking management system using Java
2. To implement Object-Oriented Programming principles for maintainable and scalable code
3. To demonstrate the use of inheritance and polymorphism through different account types
4. To implement robust exception handling for error management and system stability
5. To utilize multithreading for real-time fraud detection and analysis
6. To integrate JDBC for database operations and persistent data storage
7. To create a user-friendly web interface for seamless banking operations
8. To implement networking concepts using Java's HTTP server for client-server communication

1.4 Scope of the Project

The scope of this Banking Management System includes:

1.4.1 Functional Scope

- User registration and authentication with password encryption
- Creation and management of Savings and Current accounts
- Deposit, withdrawal, and fund transfer operations
- Real-time fraud detection with multiple detection algorithms
- Comprehensive transaction history and audit trails
- Fraud alert management and resolution system

1.4.2 Technical Scope

- Backend: Java with HTTP Server for RESTful API
- Database: PostgreSQL (Supabase) with JDBC connectivity
- Frontend: HTML5, CSS3, JavaScript for responsive web interface
- Concurrent processing: Multithreading for fraud detection
- Security: Password hashing, session management, input validation

2. SYSTEM ANALYSIS

2.1 Requirements Analysis

2.1.1 Functional Requirements

The system must provide the following functionalities:

9. User Management: Registration with email and password, secure login/logout, profile management
10. Account Management: Create new accounts (Savings/Current), view account details and balance, support multiple accounts per user
11. Transaction Processing: Deposit funds with validation, withdraw funds with balance checks, transfer funds between accounts, real-time balance updates
12. Fraud Detection: Real-time transaction monitoring, multiple fraud detection algorithms, automated alert generation, fraud resolution workflow
13. Reporting: Transaction history with filtering, account statements, fraud alert dashboard

2.1.2 Non-Functional Requirements

14. Security: Password encryption using secure hashing, SQL injection prevention, session management, input validation and sanitization
15. Performance: Fast transaction processing (under 2 seconds), concurrent user support (minimum 10 simultaneous users), efficient database queries
16. Reliability: 99% uptime target, proper exception handling, data integrity maintenance, automated fraud detection
17. Usability: Intuitive user interface, responsive design for different screen sizes, clear error messages, consistent navigation
18. Maintainability: Modular code structure, comprehensive code comments, proper naming conventions, separation of concerns

2.2 Feasibility Study

2.2.1 Technical Feasibility

The project is technically feasible as it uses well-established technologies. Java provides robust support for all required features including networking, database connectivity, multithreading, and exception handling. The HTTP server implementation using com.sun.net.httpserver package is sufficient for handling RESTful API requests. PostgreSQL database with JDBC ensures reliable data persistence and ACID compliance.

2.2.2 Economic Feasibility

The project is economically viable as it uses free and open-source technologies. Java Development Kit (JDK) is free, Supabase provides free tier for database services, and the web interface uses standard HTML/CSS/JavaScript without requiring paid frameworks or libraries. The total development cost is minimal, limited to development time and computing resources.

2.2.3 Operational Feasibility

The system is operationally feasible as it provides a familiar banking interface that users can easily understand. The web-based architecture makes it accessible from any device with a web browser, eliminating the need for special software installation. The automated fraud detection reduces manual monitoring requirements.

3. SYSTEM DESIGN

3.1 System Architecture

The Banking Management System follows a three-tier architecture consisting of Presentation Layer, Business Logic Layer, and Data Access Layer. This separation of concerns ensures maintainability, scalability, and testability of the application.

3.1.1 Presentation Layer

The presentation layer consists of HTML5 pages styled with CSS3 and enhanced with JavaScript for dynamic functionality. The frontend communicates with the backend through RESTful API calls using the Fetch API. Key pages include:

- index.html: Landing page with login and registration forms
- dashboard.html: Main dashboard showing account summary and quick actions
- accounts.html: Account management interface for creating and viewing accounts
- transactions.html: Transaction processing and history viewing
- transfer.html: Fund transfer interface between accounts
- fraud.html: Fraud alert dashboard and resolution interface

3.1.2 Business Logic Layer

The business logic layer is implemented in Java and consists of several key components:

- BankingServer: Main server class that sets up HTTP endpoints and manages server lifecycle
- BankingService: Core service class orchestrating business operations
- TransactionProcessor: Handles transaction validation and processing
- FraudAnalysisEngine: Implements fraud detection algorithms
- FraudDetectionThread: Manages concurrent fraud analysis

3.1.3 Data Access Layer

The data access layer uses the Data Access Object (DAO) pattern to abstract database operations. Key DAO classes include:

- UserDAO: Manages user-related database operations (registration, authentication)
- AccountDAO: Handles account creation, retrieval, and updates
- TransactionDAO: Manages transaction records and history
- FraudAlertDAO: Stores and retrieves fraud alerts
- DatabaseConnection: Manages JDBC connection pooling and database connectivity

3.2 Database Design

The database schema consists of four main tables designed to maintain data integrity and support efficient queries:

3.2.1 Users Table

Stores user authentication and profile information.

Column Name	Data Type	Description
id	SERIAL PRIMARY KEY	Auto-increment user ID
username	VARCHAR(50) UNIQUE	Unique username for login
email	VARCHAR(100) UNIQUE	User email address
password_hash	VARCHAR(255)	Encrypted password
full_name	VARCHAR(100)	User's full name
phone	VARCHAR(15)	Contact number
created_at	TIMESTAMP	Registration timestamp
active	BOOLEAN	Account active status

3.2.2 Accounts Table

Stores account information with different types (Savings/Current).

Column Name	Data Type	Description
id	SERIAL PRIMARY KEY	Auto-increment account ID
user_id	INTEGER FOREIGN KEY	References users(id)
account_number	VARCHAR(20) UNIQUE	Unique account number
account_type	VARCHAR(20)	SAVINGS or CURRENT
balance	DECIMAL(15,2)	Current balance
created_at	TIMESTAMP	Account creation time

active	BOOLEAN	Account active status

3.2.3 Transactions Table

Records all banking transactions for audit and history tracking.

Column Name	Data Type	Description
id	SERIAL PRIMARY KEY	Transaction ID
account_id	INTEGER FOREIGN KEY	References accounts(id)
transaction_type	VARCHAR(20)	DEPOSIT/WITHDRAWAL/TRANSFER
amount	DECIMAL(15,2)	Transaction amount
to_account_id	INTEGER	Target account for transfers
description	TEXT	Transaction description
status	VARCHAR(20)	SUCCESS/FAILED/PENDING
created_at	TIMESTAMP	Transaction timestamp

3.2.4 Fraud Alerts Table

Stores fraud detection alerts for security monitoring.

Column Name	Data Type	Description
id	SERIAL PRIMARY KEY	Alert ID
account_id	INTEGER FOREIGN KEY	References accounts(id)
transaction_id	INTEGER FOREIGN KEY	References transactions(id)
alert_type	VARCHAR(50)	Type of fraud detected
description	TEXT	Alert details
severity	VARCHAR(20)	LOW/MEDIUM/HIGH/CRITICAL
status	VARCHAR(20)	PENDING/RESOLVED/DISMISSED
created_at	TIMESTAMP	Alert creation time

4. JAVA CONCEPTS IMPLEMENTATION

This project demonstrates the comprehensive application of Java programming concepts as covered in the course syllabus. Each major concept is implemented with practical examples that contribute to the overall functionality of the banking system.

4.1 Unit I: Java Programming Basics

The foundation of the project uses core Java programming concepts including variables, data types, operators, and control flow statements. Transaction validation uses if-else statements to check conditions, while loops process collections of transactions and accounts. The project demonstrates proper use of primitive types for amounts (double) and account IDs (int), along with reference types for complex objects.

4.2 Unit II: Object-Oriented Programming

OOP principles form the core architecture of the banking system:

4.2.1 Encapsulation

All model classes (User, Account, Transaction, FraudAlert) implement encapsulation by declaring fields as private and providing public getter and setter methods. This protects data integrity and allows controlled access to object properties. For example, the Account class encapsulates balance, account number, and account type, ensuring these cannot be directly modified from outside the class.

4.2.2 Inheritance

The project implements inheritance through the Account class hierarchy. The abstract Account class serves as the base class, with SavingsAccount and CurrentAccount as concrete subclasses. This demonstrates the IS-A relationship where a SavingsAccount IS-A type of Account. Each subclass inherits common properties like account number and balance while implementing specific behaviors like different interest calculations and overdraft limits.

4.2.3 Polymorphism

Polymorphism is demonstrated through method overriding in account types. The calculateInterest() and getOverdraftLimit() methods are declared as abstract in the Account class and implemented differently in SavingsAccount and CurrentAccount. At runtime, the correct implementation is called based on the actual object type, allowing the system to treat all accounts uniformly while maintaining their specific behaviors.

4.2.4 Abstraction

The Account class is declared as abstract, providing a template for specific account types. This abstraction hides complex implementation details and presents only essential features. Users interact with accounts through simple methods like deposit() and withdraw() without needing to understand the internal balance management logic.

4.3 Unit III: Interfaces, Packages, and Exception Handling

4.3.1 Interfaces

The FraudDetector interface defines a contract for fraud detection implementations. The FraudAnalysisEngine class implements this interface, providing concrete fraud detection algorithms. This demonstrates the power of interfaces in achieving abstraction and enabling multiple implementations of the same contract. The interface declares methods like analyze() and isHighRisk() that must be implemented by any fraud detection system.

4.3.2 Packages

The project is organized into logical packages that group related classes:

- com.banking: Main server and configuration
- com.banking.model: Data models (User, Account, Transaction, FraudAlert)
- com.banking.service: Business logic and DAO classes
- com.banking.fraud: Fraud detection system
- com.banking.exception: Custom exception classes
- com.banking.util: Utility classes (DatabaseConnection, PasswordUtil, JsonUtil)

4.3.3 Exception Handling

The system implements comprehensive exception handling with custom exception classes:

- InsufficientFundsException: Thrown when withdrawal amount exceeds available balance
- InvalidAmountException: Thrown for negative or zero transaction amounts
- AccountNotFoundException: Thrown when requested account doesn't exist
- DatabaseException: Thrown for database connectivity or query errors

Try-catch-finally blocks are used throughout the codebase to handle exceptions gracefully, ensuring system stability and providing meaningful error messages to users.

4.4 Unit IV: Collections, Multithreading, and Networking

4.4.1 Collections Framework

The project extensively uses Java Collections Framework:

- ArrayList: Used to store lists of transactions, accounts, and fraud alerts. Provides dynamic array functionality for managing collections that change in size.
- HashMap: Used in FraudAnalysisEngine to track transaction patterns by IP address, enabling efficient lookup and pattern detection.
- List interface: Used as return types in DAO methods, promoting flexibility and allowing easy switching between ArrayList and other List implementations.

4.4.2 Multithreading

Multithreading is a key feature of the fraud detection system. The FraudDetectionThread class extends Thread and runs concurrently with the main application. When a transaction occurs, the fraud detection analysis runs in a separate thread, allowing the main transaction flow to continue without blocking. This demonstrates the practical use of concurrent programming to improve system responsiveness and enable real-time monitoring.

The HTTP server also uses a thread pool (Executors.newFixedThreadPool(10)) to handle multiple client requests simultaneously, demonstrating concurrent request processing in a server environment.

4.4.3 Networking

The project implements networking using Java's HttpServer class (com.sun.net.httpserver). The BankingServer creates an HTTP server that listens on a specified port and handles RESTful API requests. This demonstrates socket programming concepts where the server accepts client

connections and processes HTTP requests. The server implements multiple endpoints (contexts) for different banking operations, showing practical application of client-server architecture.

4.5 Unit V: JDBC Database Connectivity

JDBC (Java Database Connectivity) is extensively used for database operations. The DatabaseConnection class manages the connection to PostgreSQL database using the DriverManager. All DAO classes (UserDAO, AccountDAO, TransactionDAO, FraudAlertDAO) use JDBC to perform CRUD operations:

- PreparedStatement: Used for parameterized queries to prevent SQL injection attacks
- ResultSet: Used to retrieve and process query results
- Transaction Management: Uses commit() and rollback() for maintaining data consistency
- Connection Pooling: Manages database connections efficiently to handle concurrent requests

5. FEATURES AND FUNCTIONALITY

5.1 User Authentication System

The authentication system provides secure user registration and login functionality. Users register with a unique username, email, password, full name, and phone number. Passwords are encrypted using SHA-256 hashing before storage, ensuring they cannot be retrieved in plain text. The login process validates credentials against the database and creates a session for authenticated users. Session management maintains user state across multiple requests.

5.2 Account Management

Users can create and manage multiple bank accounts. The system supports two account types:

- Savings Account: Earns interest (default 4% annually) on the balance, has no overdraft facility
- Current Account: No interest earned, provides overdraft facility up to a specified limit (default ₹10,000)

Each account is assigned a unique 12-digit account number automatically. Users can view all their accounts with current balances and account details on a single dashboard.

5.3 Transaction Processing

5.3.1 Deposit

Users can deposit funds into their accounts. The system validates the deposit amount (must be positive), updates the account balance in the database, and records the transaction in the transactions table. The deposit is processed atomically to ensure data consistency.

5.3.2 Withdrawal

Withdrawal operations check if sufficient funds are available (including overdraft limit for current accounts) before processing. If funds are insufficient, an InsufficientFundsException is thrown and the transaction is rejected. Successful withdrawals update the account balance and create a transaction record.

5.3.3 Fund Transfer

Users can transfer funds between accounts using account numbers. The system verifies both source and destination accounts exist, checks for sufficient funds, and performs the transfer as a database transaction. If any step fails, the entire transfer is rolled back to maintain consistency. Both debit and credit transactions are recorded for audit purposes.

5.4 Fraud Detection System

The intelligent fraud detection system is a standout feature that analyzes transactions in real-time using multithreading. The system implements multiple fraud detection algorithms:

19. Large Transaction Detection: Flags withdrawals or transfers exceeding ₹50,000 (configurable threshold). Transactions over ₹100,000 are marked as critical priority.
20. Rapid Transaction Detection: Monitors for 5 or more transactions within a 10-minute window, which could indicate account compromise or automated fraud.
21. Unusual Hour Detection: Flags transactions occurring between 12:00 AM and 4:00 AM as potentially suspicious, as legitimate banking typically doesn't occur during these hours.
22. Round Amount Pattern Detection: Identifies repeated transactions in exact round amounts (multiples of 1000), which is a common pattern in money laundering and fraud schemes.
23. High-Frequency IP Detection: Tracks transaction patterns by IP address to detect automated attacks or bot-driven fraud attempts.

When fraud is detected, the system generates alerts with severity levels (LOW, MEDIUM, HIGH, CRITICAL) and stores them in the database. Users can view all fraud alerts on a dedicated dashboard and mark them as resolved or dismissed after investigation.

5.5 Transaction History and Reporting

The system maintains comprehensive transaction history for all accounts. Users can view their complete transaction history with details including date, time, transaction type, amount, status, and description. The transaction history can be filtered by account and provides a clear audit trail of all banking activities.

6. IMPLEMENTATION DETAILS

6.1 Development Environment

- Programming Language: Java (JDK 11 or higher)
- IDE: IntelliJ IDEA / Eclipse / VS Code
- Database: PostgreSQL 14 (Supabase cloud hosting)
- JDBC Driver: PostgreSQL JDBC Driver 42.x
- Frontend: HTML5, CSS3, Vanilla JavaScript
- Version Control: Git
- Deployment: Railway (backend) and Vercel (frontend)

6.2 Project Structure

The project follows a well-organized directory structure separating backend and frontend components:

backend/ src/ main/ java/ com/ banking/ BankingServer.java model/ service/ fraud/ exception/ servlet/ util/ frontend/ css/ js/ pages/ index.html

6.3 API Endpoints

The backend exposes RESTful API endpoints for all banking operations:

Endpoint	Method	Description
/api/auth/register	POST	User registration
/api/auth/login	POST	User authentication
/api/accounts	GET/POST	Get accounts or create new account
/api/deposit	POST	Deposit funds
/api/withdraw	POST	Withdraw funds
/api/transfer	POST	Transfer funds between accounts
/api/transactions	GET	Get transaction history
/api/fraud/alerts	GET	Get fraud alerts
/api/fraud/resolve	POST	Resolve fraud alert

6.4 Security Measures

24. Password Encryption: All passwords are hashed using SHA-256 algorithm before storage
25. SQL Injection Prevention: PreparedStatements are used for all database queries
26. Input Validation: All user inputs are validated on both client and server sides
27. Session Management: Secure session tokens for maintaining authenticated state
28. CORS Configuration: Cross-Origin Resource Sharing headers for API security
29. Error Handling: Proper exception handling prevents information leakage

7. TESTING AND VALIDATION

7.1 Testing Methodology

The banking system was thoroughly tested using multiple testing approaches to ensure reliability, security, and correctness of all features.

7.1.1 Unit Testing

Individual components were tested in isolation to verify their correct behavior. Key test cases included:

- Account class methods (deposit, withdraw) with valid and invalid amounts
- Password hashing and verification logic
- Fraud detection algorithms with various transaction patterns
- Exception handling for boundary conditions

7.1.2 Integration Testing

Integration tests verified the interaction between different system components:

- Database connectivity and JDBC operations
- API endpoints with database persistence
- Frontend-backend communication through API calls
- Fraud detection system with transaction processing

7.1.3 System Testing

End-to-end testing validated complete user workflows:

- User registration, login, account creation, and logout flow
- Complete transaction cycle from initiation to completion
- Fraud detection and alert resolution workflow
- Concurrent user sessions and transaction processing

7.2 Test Results

All test cases passed successfully. The system correctly handles normal operations, edge cases, and error conditions. Exception handling properly catches and reports errors without crashing the application. The fraud detection system accurately identifies suspicious patterns based on configured thresholds. Database transactions maintain ACID properties ensuring data consistency.

8. FUTURE ENHANCEMENTS

Based on feedback from the faculty guide and analysis of real-world banking requirements, the following enhancements are planned for future versions:

8.1 Enhanced Account Creation Process

The account creation process will be enhanced to collect comprehensive KYC (Know Your Customer) information similar to real banking systems:

- Personal Details: Date of birth, gender, marital status, nationality
- Address Information: Complete residential address with PIN code, permanent address
- Identity Proof: Aadhaar number, PAN card number, passport details
- Employment Details: Occupation, employer name, annual income range
- Nominee Information: Nominee name, relationship, contact details

8.2 Advanced Input Validation

8.2.1 Name Validation

Implement strict validation rules for names to ensure only legitimate names are accepted:

- Reject random character combinations (e.g., 'abcd', 'xyz123', 'test')
- Validate against a dictionary of common words to prevent joke entries
- Ensure proper capitalization (first letter uppercase)
- Minimum and maximum length requirements (e.g., 2-50 characters)
- Allow only alphabets, spaces, and specific characters (apostrophes, hyphens)

8.2.2 Email Verification with OTP

Implement email verification to prevent registration with fake or invalid email addresses:

- Generate a 6-digit OTP (One-Time Password) upon registration
- Send OTP to the provided email address using JavaMail API
- User must enter correct OTP within 10 minutes to complete registration
- Mark email as verified in database upon successful verification
- Restrict account operations until email is verified

8.2.3 Mobile Number Verification

Similar to email verification, implement SMS-based OTP for mobile number validation:

- Integrate SMS gateway API (e.g., Twilio, MSG91) for sending OTPs
- Validate phone number format and ensure it's active
- Rate limiting to prevent OTP spam

8.3 Proactive Fraud Prevention System

The most significant enhancement will transform the fraud detection from a post-transaction alert system to a proactive prevention system:

8.3.1 Transaction Confirmation Workflow

When the fraud detection system identifies a suspicious transaction, instead of completing it and then alerting, the system will:

30. Immediately pause the transaction processing
31. Send real-time notification to the account owner via both email and SMS
32. Display transaction details with a confirmation request
33. Provide two options: 'Yes, I made this transaction' or 'No, this is fraud - Block it'
34. If user confirms, proceed with the transaction
35. If user reports fraud, immediately cancel transaction and block the account temporarily

8.3.2 Timeout Mechanism

Implement a 5-minute countdown timer for user response:

- Start 5-minute countdown when fraud alert is triggered
- Display real-time countdown to user in notification
- If no response within 5 minutes, automatically cancel the transaction
- This protects users who might be unavailable (busy, sleeping, etc.) from fraudulent transactions
- Scammers cannot complete transactions if the legitimate user doesn't respond

8.3.3 Implementation Approach

Technical implementation will include:

- Create a pending_transactions table to hold transactions awaiting confirmation
- Implement a scheduled task (Timer/ScheduledExecutorService) to check for timeouts
- Use WebSocket or Server-Sent Events for real-time notifications to frontend
- Integrate SMS API for mobile notifications
- Add transaction status states: PENDING_CONFIRMATION, CONFIRMED, CANCELLED, TIMEOUT

8.4 Local Database Synchronization

Implement a hybrid database architecture to maintain both cloud and local database copies:

- Install PostgreSQL database locally on development machine
- Create identical schema on local database matching Supabase structure
- Implement database triggers or application-level hooks to replicate changes
- Every INSERT, UPDATE, DELETE operation on Supabase is also executed on local database
- Use database replication or implement custom sync service
- Handle conflicts and ensure data consistency between databases

8.5 Additional Planned Features

- Two-Factor Authentication (2FA) for enhanced security
- Transaction limits and daily withdrawal caps
- Loan and credit card management modules
- Automated bill payment and recurring transactions
- Account statement generation in PDF format
- Transaction analytics and spending insights
- Mobile banking application using Android/iOS
- Chatbot for customer support using AI

9. CONCLUSION

The Banking Management System successfully demonstrates the comprehensive application of Java programming concepts in building a real-world web application. The project effectively implements all major topics covered in the Java Programming course syllabus including Object-Oriented Programming, Exception Handling, Multithreading, Collections Framework, Networking, and JDBC database connectivity.

The system provides a complete banking solution with user authentication, multiple account types, transaction management, and an intelligent fraud detection system. The use of inheritance and polymorphism through the Account class hierarchy demonstrates solid understanding of OOP principles. The implementation of custom exceptions ensures robust error handling and system stability. The fraud detection system showcases practical application of multithreading for concurrent processing, while the HTTP server demonstrates networking and client-server architecture concepts.

One of the key strengths of this project is its practical relevance. The banking system simulates real-world banking operations and incorporates security best practices such as password encryption, SQL injection prevention, and input validation. The fraud detection feature adds significant value by providing real-time monitoring of suspicious activities, which is crucial in modern banking systems.

The project structure follows industry best practices with proper separation of concerns through the use of packages. The DAO pattern for database operations, service layer for business logic, and separate model classes create a maintainable and scalable architecture. This organization makes the codebase easy to understand, modify, and extend.

Throughout the development of this project, valuable lessons were learned about software engineering practices, database design, API development, and security considerations. The challenges faced during implementation, particularly in managing concurrent transactions and ensuring data consistency, provided practical insights into real-world application development.

The future enhancements planned for this system, especially the proactive fraud prevention mechanism with transaction confirmation and timeout features, will make it even more aligned with real banking systems. The addition of email and mobile verification, enhanced KYC process, and local database synchronization will transform this from an academic project into a production-ready application.

In conclusion, this Banking Management System successfully achieves its objectives of demonstrating Java programming skills while building a practical, functional banking application. It serves as a strong foundation for understanding enterprise application development and provides valuable portfolio material for demonstrating technical capabilities to potential recruiters. The project not only fulfills the academic requirements but also provides hands-on experience with technologies and concepts that are directly applicable in professional software development.

10. REFERENCES

36. Balagurusamy E., "Programming with Java", McGraw Hill Education, 5th Edition
37. Herbert Schildt, "Java: The Complete Reference", McGraw Hill Education, 11th Edition
38. Uttam K. Roy, "Advanced Java Programming", Oxford University Press
39. Oracle Java Documentation: https://docs.oracle.com/en/java/
40. NPTEL Java Programming Course: https://nptel.ac.in/courses/106/102/106102064/
41. PostgreSQL Official Documentation: https://www.postgresql.org/docs/
42. JDBC Tutorial: https://docs.oracle.com/javase/tutorial/jdbc/
43. Supabase Documentation: https://supabase.com/docs
44. Java HTTP Server Documentation: https://docs.oracle.com/en/java/javase/11/docs/api/jdk.httpserver/
45. Java Multithreading Tutorial: https://docs.oracle.com/javase/tutorial/essential/concurrency/
46. Java Collections Framework Guide: https://docs.oracle.com/javase/tutorial/collections/
47. RESTful API Design Best Practices: https://restfulapi.net/
48. Fraud Detection in Banking Systems - Research Papers from IEEE Xplore
49. Java Security and Cryptography: https://docs.oracle.com/en/java/javase/11/security/
50. Virtual Lab - Java Programming: https://java-iitd.vlabs.ac.in/